

Lab7 Report: Containerization with Docker

Course: Introduction to Cloud Computing

Student: Linjia Guo

NUIST-ID:202383910005

SETU-ID:20109903

Basic Info

Experiment Topic	Packaging and running a personal web page with Docker
Platform	GitHub Codespaces
Tools	Docker, Nginx Alpine, Git, GitHub

1. Experimental Purpose

1. Package self-written index.html static web page into a standard Docker image.
2. Master the core concept of immutable infrastructure based on fixed image snapshot.
3. Complete host-container port mapping configuration, map host port 8080/8081 to container inner port 80.
4. Observe container file isolation, low resource consumption, service horizontal scaling and port conflict rules, finish container cleanup and Git code submission.

2. Experimental Process

2.1 Verify the Docker Environment

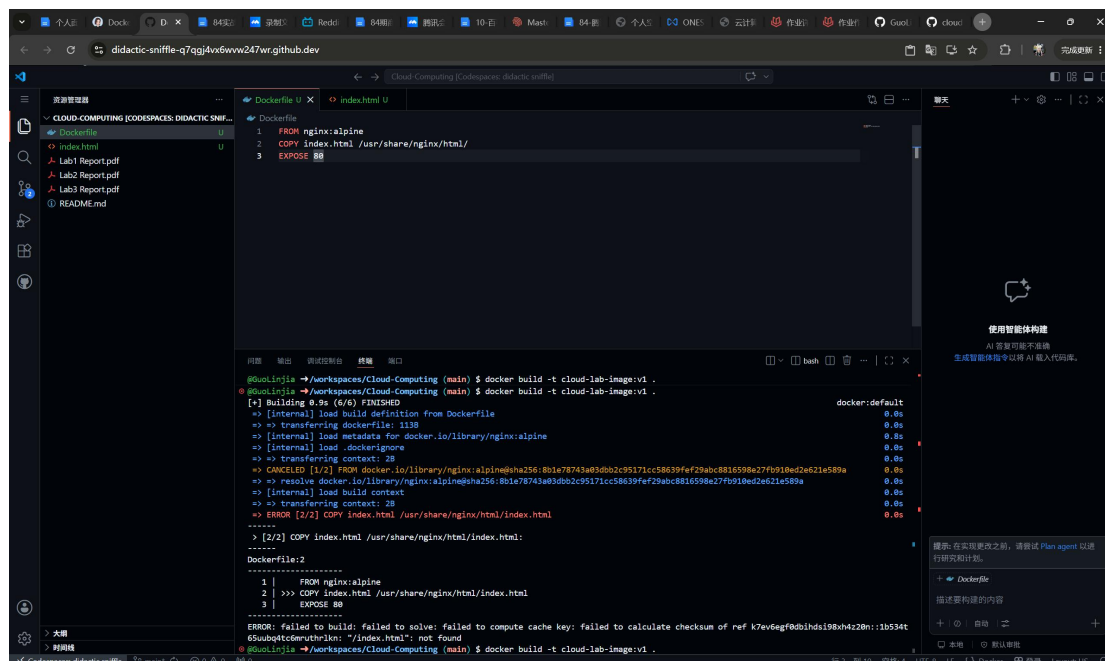
Open GitHub Codespaces terminal, input command `docker version` and execute. The terminal outputs Client and Server version information of Docker, which proves the Codespaces cloud environment has available Docker runtime normally.

```
@GuoLinJia →/workspaces/cloud-computing-lab-github (main) $ docker version
Client:
Version:      29.3.0-1
API version:  1.54
Go version:   go1.26.0
Git commit:   5927d80c76b3ce5cf782be818922966e8a0d87a3
Built:        Fri Mar  6 18:50:47 2026
OS/Arch:      linux/amd64
Context:      default

Server:
Engine:
Version:      29.3.0-1
API version:  1.54 (minimum version 1.40)
Go version:   go1.26.0
Git commit:   83bca512aa7ffc1bb4f37ce1107e0d3e3489ad43
Built:        Fri Mar  6 18:29:13 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      2.2.1-1
GitCommit:    dea7da592f5d1d2b7755e3a161be07f43fad8f75
runc:
Version:      1.4.0-2
GitCommit:    8bd78a9977e604c4d5f67a7415d7b8b8c109cdc4
docker-init:
Version:      0.19.0
GitCommit:    
```

2.2 Create the Dockerfile

Create a new file named `Dockerfile` (no suffix, case-sensitive) in the workspace root directory, write the configuration content: base on `nginx:alpine` image, copy local `index.html` into Nginx default website directory and declare exposed port 80. The Dockerfile serves as the automatic configuration script for building Docker images.



The screenshot shows a VS Code editor with a file explorer on the left containing `index.html`, `Lab1 Report.pdf`, `Lab2 Report.pdf`, `Lab3 Report.pdf`, and `README.md`. The main editor displays a `Dockerfile` with the following content:

```
1 FROM nginx:alpine
2 COPY index.html /usr/share/nginx/html/
3 EXPOSE 80
```

The terminal at the bottom shows the command `docker build -t cloud-lab-image:v1 .` being executed. The output indicates that the build failed due to a checksum error in the `COPY` instruction. The error message is:

```
ERROR: failed to build: failed to solve: failed to calculate checksum of ref k7ev6egf0db1hdsi98xh4z20n::1b534t65ub04tcc6muthr1kn: "/index.html": not found
```

2.3 Build the Docker Image

Run command `docker build -t cloud-lab-image:v1 .` in terminal to build

Docker image. After execution, the webpage and Nginx operating environment are encapsulated into an immutable image snapshot.

```
Destination could not be determined
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 5.4s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 103B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 161B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
=> => sha256:fbad3f7fcb6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 20.28MB / 20.28MB
=> => sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abff14470b5f1cfd 1.40kB / 1.40kB
=> => sha256:e654d0bbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77ad104697 1.21kB / 1.21kB
=> => sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 957B / 957B
=> => sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 405B / 405B
=> => sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 628B / 628B
=> => sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 1.88MB / 1.88MB
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
=> => extracting sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2
=> => extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd
=> => extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257
=> => extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d
=> => extracting sha256:e654d0bbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77ad104697
=> => extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abff14470b5f1cfd
=> => extracting sha256:fbad3f7fcb6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18
=> [2/2] COPY index.html /usr/share/nginx/html/
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:b9f79e09230f7f2b5b51c77da256b0efc097722e24a17a1e1c32db2a6b42f7f5
=> => exporting config sha256:3e1274a2a8219fb61bd023c4476eeb0fc1333db9f5614d69eee96c20f85bc0dc
=> => exporting attestation manifest sha256:f0609bdce5ed7bf27a3e8a79d40a7b3421f251c339ea7c32640206124e60ad4
=> => exporting manifest list sha256:91e9283c4db1ca4b8b91112c50db5dd93651945c9c1f676ba7874a2541b4c9ee
=> => naming to docker.io/library/cloud-lab-image:v1
=> => unpacking to docker.io/library/cloud-lab-image:v1
```

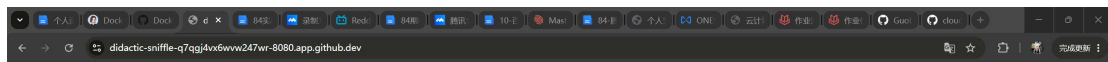
2.4 Run the First Container

Execute `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1 -p 8080:80` realizes port forwarding: access host 8080 port to reach Nginx service inside container port 80.

```
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
b26d17d03b6ec6ac197c1aa1392a00306e6b6734437b9442db20c6bc81bd3e52
```

2.5 Access the Web Server

Find Ports panel in Codespaces, open the forwarding link of port 8080 via browser, and the self-made html page loads successfully, meaning the webpage is provided by Nginx inside Docker container instead of local source file.

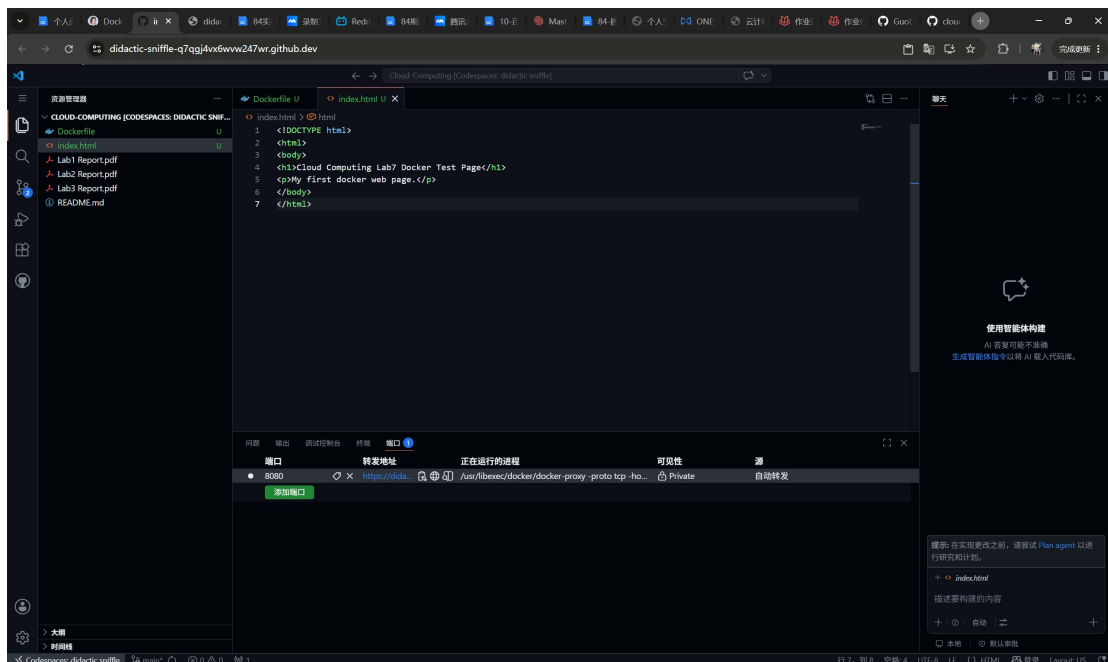


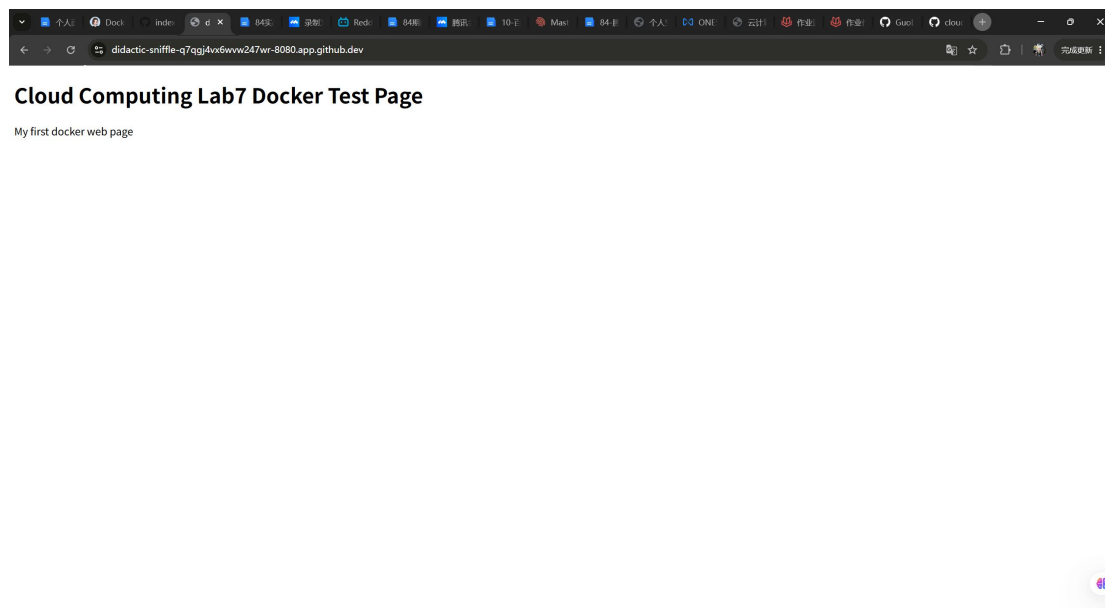
Cloud Computing Lab7 Docker Test Page

My first docker web page

2.6 Test Immutable Infrastructure

Modify the text content or background color of local index.html file in Codespaces editor, save changes. Refresh the 8080 webpage again, the webpage still displays original unchanged content. The phenomenon verifies immutable infrastructure: running container depends on the fixed image built before modification, local source file changes cannot take effect on running containers.





2.7 Explore the Container Interior

Input `docker exec -it my-web-container sh` to enter the internal shell of running container, run `ls /usr/share/nginx/html/` and `cat /usr/share/nginx/html/index.html` to check the html file stored inside container. The file content inside container is consistent with the content when building image, proving container owns independent isolated file system separated from Codespaces host directory. Type `exit` to quit container shell.

```
问题 输出 调试控制台 终端 端口 ⓘ
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker build -t cloud-lab-image:v1 .
=> => unpacking to docker.io/library/cloud-lab-image:v1 0.8s
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
b26d17d03b6ec6ac197c1aa1392a08306e6b6734437b9442db20c6bc81bd3e52
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html  index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<body>
<h1>Cloud Computing Lab7 Docker Test Page</h1>
<p>My first docker web page</p>
</body>
</html>/ #
/ #
/ #
```

2.8 Monitor Resource Usage

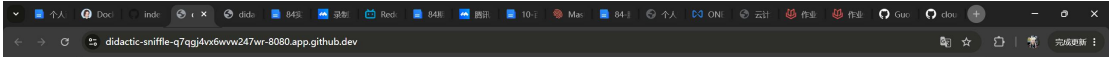
Execute `docker stats` in terminal to check real-time CPU and memory usage of running container. The idle Nginx container occupies extremely low memory and near-zero CPU usage, which reflects containers share host OS kernel and have far higher resource utilization efficiency than traditional virtual machines. Press `Ctrl+C` to exit monitoring.

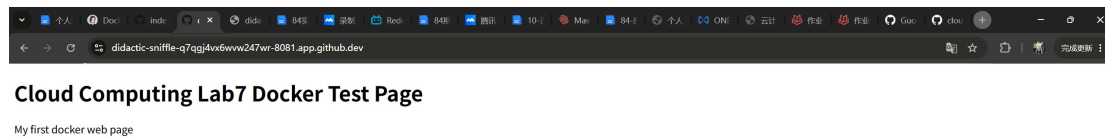
```
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
b26d17d03b6e  my-web-container  0.00%    3.434MiB / 7.758GiB   0.04%    6.84kB / 3.46kB   20.5kB / 24.6kB   3
```

2.9 Start a Second Container

Run `docker run -d -p 8081:80 --name second-container cloud-lab-image:v1` to start the second container based on the same image with host port 8081. Check Codespaces Ports panel, both 8080 and 8081 ports are in running state. Open two ports separately in browser, two identical web pages appear, realizing simple horizontal scaling of cloud service.

端口	转发地址	正在运行的进程	可见性	源
8080	https://didactic-sniffle-...	/usr/libexec/docker/docker-proxy -proto tcp -ho...	Private	自动转发
8081	https://didactic-sniffle-...	/usr/libexec/docker/docker-proxy -proto tcp -ho...	Private	自动转发





2.10 Create a Port Conflict

Attempt to create the third container using occupied host port 8080 with the same run command. Docker outputs port occupied error prompt, explaining same host port can only bind one container process, reasonable port planning is required for multi-container deployment.

```
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker run -d -p 8080:80 --name test cloud-lab-image:v1
2b3e74be54aba784260db58f683c5ed2ff6842d015e27b63c6f9ef178b6bbaf4
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint test
(19e18b440c6f4f882c1318ebd7d5f18afaf5fab2b9b009019fe277fdaa4f2db7): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
```

2.11 Clean Up Containers

Run `docker stop my-web-container second-container` to stop running containers, then execute `docker rm my-web-container second-container` to delete container instances. Input `docker ps` to verify zero active containers left. Timely cleaning idle resources conforms to cloud pay-as-you-go cost-saving specification.

```
Run 'docker run --help' for more information
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker stop my-web-container second-container
my-web-container
second-container
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker rm my-web-container second-container
my-web-container
second-container
@GuoLinJia →/workspaces/Cloud-Computing (main) $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

2.12 Save the Lab with Git

Execute sequentially:

```
git add .
```

```
git commit -m "Finished Lab 02"
```

```
git push
```

If push failure occurs due to remote branch version conflict, run `git pull --rebase origin main` first to synchronize code, then push again to store all experiment files permanently on GitHub remote warehouse.

```
@GuoLinJia →/workspaces/Cloud-Computing (main) $ git add .
@GuoLinJia →/workspaces/Cloud-Computing (main) $ git commit -m "Finished Lab 02"
[main 710d737] Finished Lab 02
 2 files changed, 10 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 index.html
@GuoLinJia →/workspaces/Cloud-Computing (main) $ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 496 bytes | 496.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/GuoLinJia/Cloud-Computing
 4858e6b..710d737  main -> main
```

3. Experimental Summary

1. Docker realizes application + operating environment one-stop packaging via image file, avoids repetitive manual environment configuration and supports consistent deployment in any environment.
2. Port mapping separates host network and container internal network, users access service through host exposed port, while internal program always listens fixed container port 80.
3. Immutable feature is core advantage of cloud deployment: container runs based on fixed image snapshot, local source file changes cannot modify online service, effectively avoid unexpected online environment mutation and improve system stability.
4. Container shares host operating system kernel, memory and CPU consumption are extremely low compared with traditional virtual machine, greatly reducing cloud server resource cost and supporting large-scale batch deployment.
5. Same Docker image can rapidly generate multiple service copies to realize horizontal scaling, but different containers must occupy different host ports, otherwise port binding conflict will appear.
6. This experiment integrates multiple cloud core knowledge including immutable infrastructure, container isolation, resource optimization, elastic scaling and Git version management.

4. Problems Encountered and Solutions

1. Codespaces access link fails after long idle: Codespaces port forwarding is temporary, close and reopen Codespaces workspace, restart container and open port from Ports panel again to fix access failure.
 2. Modified html page cannot refresh update: It is normal immutable feature instead of error, solve by rebuilding Docker image and restart container.
 3. Git push command report non-fast-forward error: Remote warehouse has newer submitted files, execute `git pull --rebase origin main` to synchronize local and remote code before pushing code.
-